

Discussion Unit:

DSE Java Driver; Use, Key Concepts

Introduce the DSE Java Driver, a Primer

- Compile and run a DSE Java client
- Introduce some of the capabilities of the driver
- Load balancing policies
- Failures: Retry, Reconnect
- Using Connections, Sessions, Statements, (other) ...
- *This Discussion Unit gives focus to programming against DSE Core*

Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

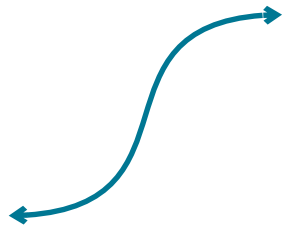
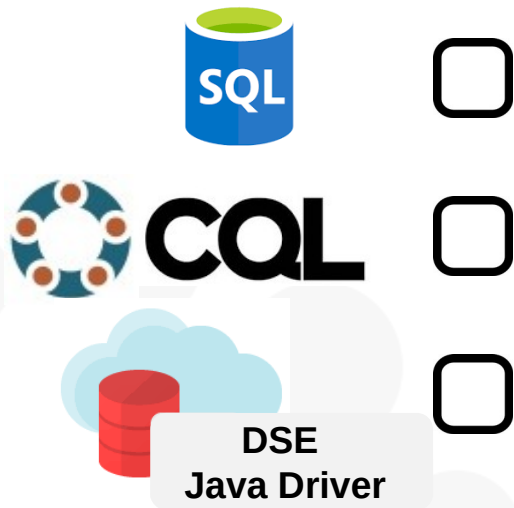


CQL



**DSE
Java Driver**

Discussion Lab: Attributes



- ☐ Declarative
- ☐ Fluent
- ☐ JDBC (JCA compliant)
- ☐ Asynchronous
- ☐ Fault Tolerant
- ☐ Connection Pools
- ☐ Query Builder
- ☐ Transactions

End of Discussion Lab:

[All] DSE Client Side Drivers-

Consistent set of features across languages

- Asynchronous execution
- Automatic cluster mapping
- Connection pools, auto reconnect
- Load balancing
- Fault tolerant

Consistent terminology

DseCluster -> DseSession -> (Statement) -> ResultSet (or) Future

(The above displays the very simplest path.)

DSE Java Driver-

```
DseCluster my_cluster = null;
```

```
my_cluster = DseCluster.builder()  
    .addContactPoint("127.0.0.1")  
    .build();
```

```
DseSession my_session = my_cluster.connect();
```

```
Row my_row = my_session.execute(  
    "select * from system.local").one();
```

```
int col1 = my_row.getInt("col2")
```

- Connect to DSE cluster
- Similar to JDBC/SQL
- Not JCA compliant

DSA 420 Java Programming: 2 Days

Part 1- The Driver

- Base cluster object
- Configuration options

Part 2- Execution

- Executing CQL
- Statements, how/when

Part 3- Annotations, Mapping

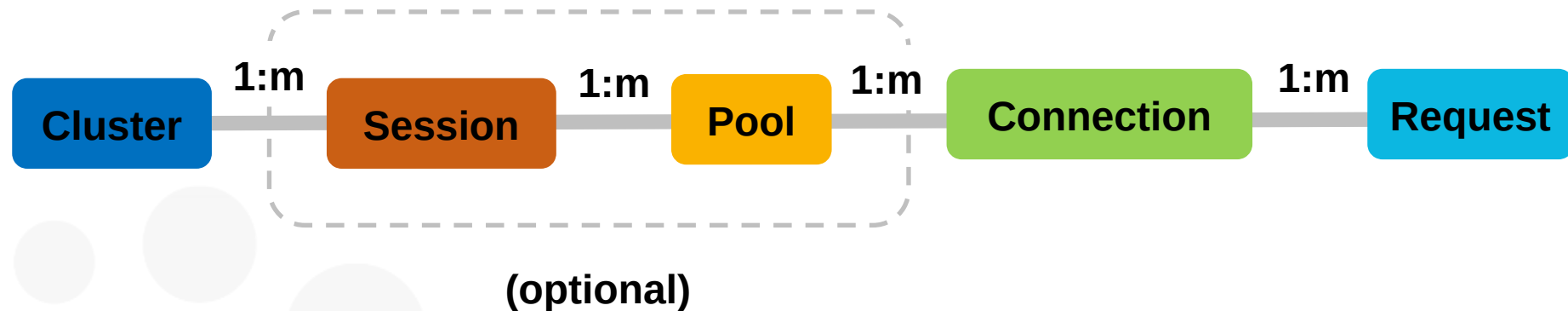
- Simple object mapping
- JPA like interface

Part 4- (Other)

- Time series
- Tombstones
- Transactions



DSE Java Driver: DseCluster object



- Load balancing
- Failover
- Connection pooling
- (other)

DseCluster.builder: Fluent Interface

```
DseCluster my_cluster = DseCluster.builder()
```

```
.addContactPoints("127.0.0.1", ..
```

```
.withLoadBalancingPolicy( ... )
```

```
.withRetryPolicy( ... )
```

```
.withReconnectPolicy( ... )
```

```
.withPoolingOptions( ... )
```

```
.withQueryOptions( ... )
```

```
.build()
```

These will default out

Load Balancers-

- Used to choose a coordinator
- Maintains pulse on node up/down/other state
- (Downed nodes excluded from query plans)
- Chainable, nestable (just like DSE Search analyzers)
- Default,
 `new TokenAwarePolicy(new DCAwareRoundRobinPolicy("local-dc-name") )`



Load Balancers-

Non-chainable policies (inner policies)-

- DCAwareRoundRobinPolicy (optional DC)
- RoundRobinPolicy



Chainable policies (outer policies)-

- TokenAwarePolicy
- LatencyAwarePolicy (define acceptable latencies)
- WhiteListPolicy
- HostFilterPolicy (super of WhiteListPolicy)

Load Balancer, Examples-

```
DseCluster my_cluster = DseCluster.builder()  
    .withLoadBalancingPolicy(tokenAware)  
    .build()
```

```
DseCluster my_cluster.DseCluster.builder()  
    .addContactPoints("127.0.0.1", ..  
    .withLoadBalancingPolicy(whiteListPolicy(["127.0.0.1", .. ]))  
    .build();
```

```
LatencyAwarePolicy my_lap = LatencyAwarePolicy.builder(  
    new RoundRobinPolicy())  
    .withMinimum.Measurements(5)  
    .build()
```

```
DseCluster my_cluster = DseCluster.builder()  
    .withLoadBalancingPolicy(my_lap)  
    .build()
```

Failover and Recovery; Retry, Reconnection Policies

Retry

- `onReadTimeout()`, `onWriteTimeout()`, `onUnavailable()`
- Whether a retry is made, what the consistency level should be
- Available policies
 - `DefaultRetryPolicy`
 - `DowngradingConsistencyRetryPolicy`
 - `FallthroughRetryPolicy`
 - `LoggingRetryPolicy`

Failover and Recovery; Retry, Reconnection Policies

ConstantReconnectionPolicy

Constant/configurable time between each attempt

ExponentialReconnectionPolicy

Wait exponentially longer between each attempt
(up to configurable max)

Control Host-

At driver level, special connection to single node

- Any node, will change if current goes down
- Host sends notice of; schema change, token assignment, node health, other

Push Notifications

Driver can receive distinct notice of a node changing to UP, other

(Connection) Pooling-

```
DseCluster my_cluster = DseCluster.builder()  
    .addContactPoints( ... )  
    .withPoolingOptions(new PoolingOptions)  
        .setConnectionsPerHost(HostDistance.LOCAL, 1, 4)  
        .setMaxRequestsPerConnection(HostDistance.LOCAL, 800)  
        .setNewConnectionThreshold(HostDistance.LOCAL, 100))  
    .build()
```


DseSession-

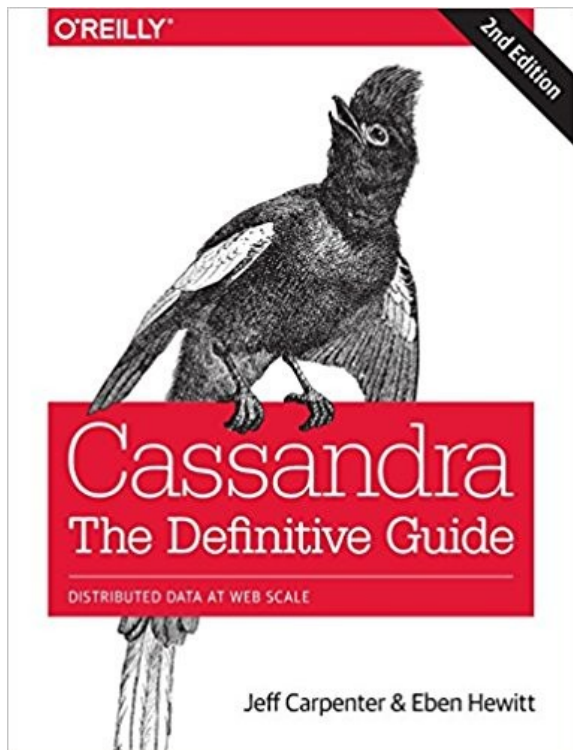
```
DseSession my_session = my_cluster.connect( <keyspace name> )
```

- Both DseCluster and DseSession are singleton objects
- One per DSE Core workload and one per DSE Search ?
- Or, tune using cassandra-stress. Increase until speed goes negative

Now that you are
connected-



Sourcing: O'Reilly Book (2016)



The example code that follows arrive from the O'Reilly Book,
"Cassandra: the Definitive Guide"

and are publicly downloadable from,
<https://github.com/jeffreyscarpenter/cassandra-guide>

The examples here are written contiguously,
with objects referenced page over page-

Executing CQL-

```
my_session.execute("SELECT * FROM t1");
```

Or ,statements-

SimpleStatement

BoundStatement

PreparedStatement

BatchStatement

(same as above, how variables are treated)

Efficiency, compiled/cached

.setConsistencyLevel()

.enableTracing()

.setFetchSize()

(others)

Example Statements: Base runtime (CQL objects)

```
CREATE TABLE ks_6240.cust_orders
(
  region          TEXT,
  cust_name       TEXT,
  ord_num         INT,
  other           TEXT,
  PRIMARY KEY ((region, cust_name), ord_num)
);
```

Example Statements: Imports (all)

```
import com.datastax.driver.dse.DseCluster;  
import com.datastax.driver.dse.DseSession;  
    //  
import com.datastax.driver.core.Row;  
    //  
import com.datastax.driver.core.Cluster;  
import com.datastax.driver.core.Host;  
import com.datastax.driver.core.Metadata;  
    //  
import com.datastax.driver.core.Session;
```

Example Statements: Imports (all)

```
import com.datastax.driver.core.QueryTrace;
import com.datastax.driver.core.ResultSet;
import com.datastax.driver.core.Row;
import com.datastax.driver.core.SimpleStatement;
    //
import java.text.SimpleDateFormat;
    //
import com.datastax.driver.core.BoundStatement;
import com.datastax.driver.core.PreparedStatement;
    //
import com.datastax.driver.core.BatchStatement;
```

Example Statements: Imports (all)

```
import com.datastax.driver.core.querybuilder.BuiltStatement;  
import com.datastax.driver.core.querybuilder.QueryBuilder;  
import static  
    com.datastax.driver.core.querybuilder.QueryBuilder.eq;  
    //  
import java.util.Arrays;  
import java.util.List;
```


Example Statements-

```
DseCluster my_cluster = null;

my_cluster = DseCluster.builder()
    .addContactPoint("127.0.0.1")
    // .withCredentials("guest", "password")
    .build();

DseSession my_session1 = my_cluster.connect();

Row my_row1 = my_session1.execute("select * from
system.local").one();

System.out.println("DSE release version: " +
my_row1.getString("dse_version") );
```

```
Metadata my_metadata1 = my_cluster.getMetadata();
```

```
System.out.printf("Connected to cluster: %s %s\n",  
    my_metadata1.getClusterName(),  
    my_cluster.getClusterName());
```

Example Statements-

```
for (Host host : my_metadata1.getAllHosts()) {  
    System.out.printf("Data Center: %s; Rack: %s; Host: %s\n",  
        host.getDatacenter(), host.getRack(), host.getAddress());  
}
```

```
System.out.printf("Protocol Version: %s\n",  
    my_cluster.getConfiguration()  
        .getProtocolOptions()  
        .getProtocolVersion());
```

Example Statements-

```
Session.State my_state = my_session1.getState();

System.out.printf("New session created for keyspace: %s\n",
    my_session1.getLoggedKeyspace());

for (Host host : my_state.getConnectedHosts()) {
    System.out.printf("Data Center: %s; Rack: %s; \
        Host: %s; Open Connections: %s\n",
        host.getDatacenter(), host.getRack(), host.getAddress(),
        my_state.getOpenConnections(host));
}
```

```
Session my_session2 = my_cluster.connect("ks_6240");
```

```
String my_str1 = "JC Penney";
```

```
SimpleStatement my_insert1 = new SimpleStatement(  
    "INSERT INTO cust_orders (region, cust_name,  
    ord_num, other) VALUES (?, ?, ?, ?)",  
    "NA", my_str1, 109, "Ball, Mitt");
```

Example
Statements-

```
ResultSet my_result1 = my_session2.execute(my_insert1);  
System.out.println(my_result1);  
System.out.println(my_result1.wasApplied());  
System.out.println(my_result1.getExecutionInfo());  
System.out.println(my_result1.getExecutionInfo().  
    getIncomingPayload());
```

```
SimpleStatement my_select1 = new SimpleStatement(  
    "SELECT * FROM cust_orders WHERE region = ? \  
    AND cust_name = ?", "NA", my_str1);
```

```
my_select1.enableTracing();
```

```
ResultSet my_result2 = my_session2.execute(my_select1);
```

```
System.out.println(my_result2);
```

```
System.out.println(my_result2.wasApplied());
```

```
System.out.println(my_result2.getExecutionInfo());
```

```
System.out.println(my_result2.getExecutionInfo().  
    getIncomingPayload());
```

```
System.out.println(my_result2.getExecutionInfo().  
    getQueryTrace());
```

Example Statements-

Example Statements-

```
for (Row row : my_result2) {  
    System.out.format("region: %s, cust_name: %s, \  
        ord_num: %s, other: %s\n\n",  
        row.getString("region"), row.getString("cust_name"),  
        row.getInt("ord_num"), row.getString("other") );  
}
```

```
SimpleDateFormat my_dateFormat = new  
SimpleDateFormat("HH:mm:ss.SSS");
```

```
QueryTrace my_queryTrace1 =  
my_result2.getExecutionInfo().getQueryTrace();
```

Example Statements-

```
System.out.printf("Trace id: %s\n\n",  
    my_queryTrace1.getTraceId());  
System.out.printf("%-42s | %-12s | %-10s \n",  
    "activity", "timestamp", "source");  
  
System.out.println("----- ... -----");
```

Example Statements-

```
for (QueryTrace.Event event : my_queryTrace1.getEvents()) {  
    System.out.printf("%42s | %12s | %10s\n",  
        event.getDescription(),  
        my_dateFormat.format((event.getTimestamp())),  
        event.getSource());  
}
```


Example Statements-

```
String my_str2 ="Woolworth's";
```

```
PreparedStatement my_prepared1 = my_session2.prepare(  
    "INSERT INTO cust_orders (region, cust_name,  
    ord_num, other) VALUES (?, ?, ?, ?)");
```

```
BoundStatement my_bound1 = my_prepared1.bind(  
    "NA", my_str2, 110, "Robot, Car");
```

```
ResultSet my_result3 = my_session2.execute(my_bound1);
```

Example Statements-

```
System.out.println(my_result3);  
System.out.println(my_result3.wasApplied());  
System.out.println(my_result3.getExecutionInfo());  
System.out.println(my_result3.getExecutionInfo().  
    getIncomingPayload());
```

Example Statements-

```
PreparedStatement my_prepared2 = my_session2.prepare(  
    "SELECT * FROM cust_orders WHERE region = ? \  
    AND cust_name = ?");  
BoundStatement my_bound2 = my_prepared2.bind(  
    "NA", "SEARS");  
  
ResultSet my_result4 = my_session2.execute(my_bound2);
```

Example Statements-

```
ResultSet my_result4 = my_session2.execute(my_bound2);  
  
System.out.println(my_result4);  
System.out.println(my_result4.wasApplied());  
System.out.println(my_result4.getExecutionInfo());  
System.out.println(my_result4.getExecutionInfo().  
    getIncomingPayload());
```

Example Statements-

```
for (Row row : my_result4) {  
    System.out.format("region: %s, cust_name: %s, \  
        ord_num: %s, other: %s\n",  
        row.getString("region"), row.getString("cust_name"),  
        row.getInt("ord_num"), row.getString("other"));  
}
```

Example Statements-

```
SimpleStatement my_delete1 = new SimpleStatement(  
    "DELETE FROM cust_orders WHERE region = ? \  
    AND cust_name= ?", "NA", my_str2);  
  
ResultSet my_result5 = my_session2.execute(my_delete1);  
  
System.out.println(my_result5);  
System.out.println(my_result5.wasApplied());  
System.out.println(my_result5.getExecutionInfo());  
System.out.println("num results: " +  
    my_result5.all().size());
```

Example Statements-

```
for (Row row : my_result5) {  
    System.out.format("region: %s, cust_name: %s, \  
        ord_num: %s, other: %s\n",  
        row.getString("region"), row.getString("cust_name"),  
        row.getInt("ord_num"), row.getString("other"));  
}
```

Example Statements-

```
SimpleStatement my_insert2 = new SimpleStatement(  
    "INSERT INTO cust_orders (region, cust_name, ord_num,  
    other) VALUES (?, ?, ?, ?)",  
    "SA", "X-Store", 101, "Water, Juice");  
SimpleStatement my_insert3 = new SimpleStatement(  
    "INSERT INTO cust_orders (region, cust_name, ord_num,  
    other) VALUES (?, ?, ?, ?)",  
    "SA", "X-Store", 102, "Water, Juice, Lemons");
```


Example Statements-

```
BatchStatement my_batch1 = new BatchStatement();  
my_batch1.add(my_insert2);  
my_batch1.add(my_insert3);  
  
ResultSet my_result6 = my_session2.execute(my_batch1);
```

Example Statements-

```
System.out.println(my_result6);  
System.out.println(my_result6.wasApplied());  
System.out.println(my_result6.getExecutionInfo());  
System.out.println(my_result6.getExecutionInfo().  
    getIncomingPayload());
```

Example Statements-

```
SimpleStatement my_insert4 = new SimpleStatement(  
    "INSERT INTO cust_orders (region, cust_name,  
    ord_num, other) VALUES (?, ?, ?, ?) IF NOT EXISTS",  
    "SA", "X-Store", 103, "Water, Juice, Pie");
```

```
ResultSet my_result7 = my_session2.execute(my_insert4);
```

Example Statements-

```
System.out.println(my_result7);  
System.out.println(my_result7.wasApplied());  
System.out.println(my_result7.getExecutionInfo());  
System.out.println(my_result7.getExecutionInfo().  
    getQueryTrace());
```

Example Statements-

```
for (Row row : my_result7) {  
    System.out.format("region: %s, cust_name: %s, \  
        ord_num: %s, other: %s\n",  
        row.getString("region"), row.getString("cust_name"),  
        row.getInt("ord_num"), row.getString("other") );  
}
```

Example Statements-

```
Metadata my_metadata2 = my_cluster.getMetadata();
```

```
System.out.printf("Connected to cluster: %s %s\n",  
    my_metadata2.getClusterName(),  
    my_cluster.getClusterName());
```

```
System.out.println("Schema:");  
System.out.println(my_metadata2.exportSchemaAsString());  
System.out.println();
```

```
System.out.printf("Schema agreement : %s\n",  
    my_metadata2.checkSchemaAgreement());
```

DSE Query Builder

- Nothing new [functionally] over previous content/methods
(Inherits from Statement, thus, all options available.)
- Syntactic sugar-
- Easier to read, maintain ??

Query Builder API-

- Alternative to (manual) query strings
- Generates a Statement, per earlier

- `QueryBuilder.select()`
- `QueryBuilder.insertInto()`
- `QueryBuilder.update()`
- `QueryBuilder.delete()`

- `.enableTracing()`
- `.isIdempotent()`
- `.limit()`
- `.orderBy()`
- (others)

Query Builder Example-

```
BuiltStatement my_built1 =  
QueryBuilder.insertInto("cust_orders").  
    value("region", "EMEA").  
    value("cust_name", "Apple").  
    value("ord_num", 200).  
    value("other", "mouse");
```

```
ResultSet my_result8 = my_session2.execute(my_built1);
```

```
System.out.println(my_result8);  
System.out.println(my_result8.wasApplied());
```

```
System.out.println(my_result8.getExecutionInfo());
```

```
System.out.println(my_result8.getExecutionInfo()  
    .getIncomingPayload());
```

Query Builder Example-

```
Select my_select = QueryBuilder.select()  
    .all()  
    .distinct()  
    .from( "ks", "table")  
    .where(eq("col", "value"))  
    .and(eq("col", "value"))  
    .orderBy(asc("col"))  
    .limit(2);
```

```
System.out.println(my_select.getQueryString());  
// SELECT * FROM ..
```

```
ResultSet my_result = my_session.execute(my_select);  
System.out.println(my_result.one().getString("col"));
```

```
Delete.Where my_delete = QueryBuilder.delete()  
    .from("ks", "table")  
    .where(eq("col", "value"));  
  
my_session.execute(my_delete);
```

Query Builder Example-



Topics Not Covered:

Topics Not Covered-

Futures

- DSE is *inherently asynchronous*
- All prior objects were wrappers for the default asynchronous, but calling for synchronous
- Futures require Java annotations, which we don't want to teach in this intro level unit.
- Other JCA topics-
- Other

